

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Welcome to the course!

About Me

B.Engr. Computer Science, Wuhan University, 2013-2017

Ph.D. Computer Engineering, University of Toronto,
2017-2023

- Research interest: Federated Learning

Industry Experience: 20 months as a researcher of
Computer Network

Current: Assistant Professor, Teaching Stream, since 2025

What are you learning in this course?

Developing creative and innovative mobile applications using **React Native** and **Expo**

Technologies

- **React**: JavaScript library for building **interactive** and **component-based** user interfaces
- **React Native**: Framework for building **cross-platform native apps** (iOS, Android) using React and JavaScript

Native App

Application that is **written to run directly on a specific platform** (iOS or Android) using that platform's native programming languages and APIs:

- iOS: Swift or Objective-C; UIKit or SwiftUI
- Android: Kotlin or Java; Android SDK

React Native

Build native apps using JavaScript and React

One React Native codebase

→ A native app for both iOS & Android

Mobile Development

View: Core building block of UI

- A small rectangular element on the screen which can be used to display text, images, or handle user input

Views: iOS vs. Android

iOS View	Android View
<code><UIView></code>	<code><ViewGroup></code>
<code><UITextView></code>	<code><TextView></code>
<code><UIImageView></code>	<code><ImageView></code>
<code><UIScrollView></code>	<code><ScrollView></code>
<code><UITextField></code>	<code><EditText></code>

React Native use **React components** to create views

React Native Components

React Native provides built-in Core Components

RN Component	iOS View	Android View
<View>	<UIView>	<ViewGroup>
<Text>	<UITextView>	<TextView>
<Image>	<UIImageView>	<ImageView>
<ScrollView>	<UIScrollView>	<ScrollView>
<TextInput>	<UITextField>	<EditText>

React Native Components

React Native components are automatically translated into native views

- Render native UI on both iOS and Android

```
1 import { View, Text } from "react-native";
2
3 const App = () => {
4   return (
5     <View>
6       <Text>Hello, world!</Text>
7     </View>
8   );
9 };
```

One codebase for two platforms

Technologies

- **React**: JavaScript library for building **interactive** and **component-based** user interfaces
- **React Native**: Framework for building **cross-platform native apps** (iOS, Android) using React and JavaScript
- **Expo**: Development platform that simplifies React Native app development, testing, and deployment

Prerequisites

- **JavaScript:** required
 - Variables, functions, arrays, and objects
 - ES6+ syntax: `let`, `const`, arrow functions, destructuring, modules
 - Asynchronous programming: `Promises`, `async/await`
- **TypeScript:** preferred
- **React:** preferred

Tools

- Text Editor (**VS Code** recommended)
- **Git** for version control
- **Node.js** (Latest LTS version): A JavaScript runtime to run JavaScript outside of browser
- **Expo**: A framework that makes developing iOS and Android apps easier

Create Your First App

```
npx create-expo-app@latest
```

Run the app locally

```
npx expo start
```

Tools

- **Expo Go**: A sandbox for testing Expo projects quickly on real devices
- **Android Studio**: Required to run **Android Emulator**
- **Xcode** (macOS only): Required to run **iOS Simulator** on Mac

Recommended Resources

TypeScript Handbook

React Documentation

React Native Documentation

Expo Documentation

Grading

Assignments (50%)

- 4 assignments, 12.5% each, done individually

Project (50%)

Work in groups of 2 to 4 to develop a mobile application

- **Proposal (15%)**: due on October 19, 2025, 11:59 PM
- **Presentation (10%)**: November 19 & 26
- **Final Deliverable and Report (25%)**: due on December 7, 2025, 11:59 PM

Project

- Done in groups of 2 to 4
- The topic is of your own choosing
- A mobile application implementing the technologies learned in this course
- Requirements, sample ideas, marking rubrics and more details will be on **course website**

Presentation: Nov 19 & 26

1-2 members present a ~5 minute in-class demo

- 10%: 5% by peers and 5% by instructor and TAs
- **Presentation Rubric**
- Slots will be randomized and announced by October 27
- Submit presentation slides via Quercus by November 17
- Peer scores will be normalized across Nov 19 and 26

Project Timeline

Milestone	Due Date
Team Member List	October 5
Project Proposal	October 19
Presentation Slides	November 17
Presentation	November 19 & 26
Final Project Deliverable	December 7

Grading

Assignments (50%)

Project (50%)

Bonus Points

- In-class quizzes will be held in some lectures
- If a quiz is scheduled, students will be notified in the previous lecture
- Each quiz only covers material from that lecture

Teaching Assistants

Yining Wang: yining.wang@mail.utoronto.ca

Yiren Zhao: yiren.zhao@mail.utoronto.ca

Office Hours

- Time: Thursdays from 2:00 PM to 3:00 PM
- Location: Room 7206, Bahen Centre for Information Technology
- First Office Hour: September 11, 2025
- Last Office Hour: November 27, 2025

Three Websites

1. Quercus

- Assignment submission
- Grades
- Announcements

2. Course website

- Slides, video recordings of lectures, handouts

3. GitHub Discussions

- Submit your GitHub username via this assignment on Quercus

Today's Lecture

React Refresher

Introduction to Mobile Development with React Native and Expo

React

A JavaScript Library for building interactive user interfaces

- **Declarative Approach:** You describe *what the UI should look like*, not *how to update it step by step*
- Define **components** and build UI with components

Component

The building block of React applications

- A piece of the UI that has its own logic and appearance
- Written as a JavaScript **function** that returns **JSX**

```
1 function MyButton() {  
2   return <button>Click me</button>;  
3 }
```

JSX

JSX (JavaScript XML): A syntax extension for JavaScript that allows you to write HTML-like code within JavaScript

- Combines the power of JavaScript with the readability of HTML

Component: Example

```
1 function MyButton() {  
2   return <button>Click me</button>;  
3 }
```

- `MyButton` is a component
- `<MyButton />` can be used just like `<button />`

```
1 const App = () => {  
2   return <MyButton />;  
3 };
```

Props

Stands for properties

- Used to pass data from a parent component to a child component
- **Read-only**: A child component cannot change received prop

Prop: Example

```
1  const MyButton = ({ label }) => {  
2    return <button>{label}</button>;  
3  };  
4  
5  const App = () => {  
6    return (  
7      <div>  
8        <MyButton label="Click me" />  
9        <MyButton label="Submit" />  
10     </div>  
11   );  
12  };
```

- Parent component `App` passes prop `label` to child component `MyButton`

Props make components reusable and flexible

Event Handling

React lets components respond to user input (clicks, typing, etc.)

- Pass handlers (e.g., `onClick`) as props to components

```
1 import { useState } from "react";
2
3 const MyButton = () => {
4   const [count, setCount] = useState(0);
5
6   return (
7     <button onClick={() => setCount(count + 1)}>Clicked {count} times</bu
8   );
9 };
```

- `useState` makes the button “remember” its state across clicks

Live Demo

react-example/src/App.jsx

```
1 import { useState } from "react";
2
3 const MyButton = () => {
4   const [count, setCount] = useState(0);
5
6   return (
7     <button onClick={() => setCount(count + 1)}>Clicked {count} times</bu
8   );
9 };
10
11 const App = () => {
12   return <MyButton />;
13 };
14
15 export default App;
```

- `useState` makes the button “remember” its state across clicks

Hooks

Special functions that add features to components

- `useState`: Manage component state
- `useEffect`: Handle side effects (e.g., fetch data, update page title)

Example: Create a `Counter` component that

- Displays a button which increments count when clicked
- `useState` stores and updates the current count
- Updates the page title whenever the count changes
- `useEffect` updates the page title

Hook: Example

```
1 import { useState, useEffect } from "react";
2
3 const Counter = () => {
4   const [count, setCount] = useState(0);
5
6   useEffect(() => {
7     document.title = `Count: ${count}`;
8   }, [count]);
9
10  return <button onClick={() => setCount(count + 1)}>Count: {count}</butt
11  };
12
13 const App = () => {
14   return <Counter />;
15  };
```

Live Demo

| react-example/src/App.jsx

```
1 import { useState, useEffect } from "react";
2 import "./App.css";
3
4 const Counter = () => {
5   const [count, setCount] = useState(0);
6
7   useEffect(() => {
8     document.title = `Count: ${count}`;
9   }, [count]);
10
11   return <button onClick={() => setCount(count + 1)}>Count: {count}</butt
12 };
13
14 const App = () => {
15   return <Counter />;
16 };
17
18 export default App;
```

React Hooks

`useState`: Manage component state

`useEffect`: Handle side effects (e.g., update page title)

Official Doc: [**react.dev/reference/react/hooks**](https://react.dev/reference/react/hooks)

Why React?

Without React: Manual DOM manipulation (e.g., `document.querySelector`)

- Error-prone and complex

With React: Declarative UI updates based on state changes

```
useEffect(() => {  
  document.title = `Count: ${count}`;  
}, [count]);
```

Why React?

Without React: Manual DOM manipulation (e.g., `document.querySelector`)

- Error-prone and complex

With React: Declarative UI updates based on state changes

- You declare the UI as a function of state
- React updates the DOM automatically when state changes

Recap: React Refresher

React: JavaScript library for building component-based UIs

- **Components:** Reusable pieces of UI, defined as functions returning JSX (HTML-like syntax in JavaScript)
- **Props:** Pass data to components (read-only)
- **Event Handling:** Handle interactions with event handlers like `onClick`
- **Hooks:** Manage state (`useState`) and side effects (`useEffect`)

Resource: Learn React

Introduction to Mobile Development with React Native and Expo

React Native

JavaScript-based framework for building native-like apps for iOS and Android

- Large community, reusable code across iOS/Android
- Integrates with device sensors (camera, GPS, accelerometer, etc.)
- Backed by Meta, used by apps like Facebook, Instagram

React Native

Use the same component-based approach as React

- JSX, props, state, and hooks are directly applicable

Differences: **React Native components** (e.g., `<View>`, `<Button>`) instead of HTML

```
1 import { Button } from "react-native";
2
3 <Button title="Click me" onPress={() => alert("Hello!")} />
```

- `onPress` prop of `<Button>` component is set to an arrow function that calls `alert("Hello!")`
- Trigger a native alert dialog when the button is pressed

Expo

A production-grade platform for simplifying React Native development

- Free, open source with an active community on **GitHub** and **Discord**
- Docs: **docs.expo.dev/**

Key Features:

- **Expo CLI**: Easy project setup and management
- **Expo Go**: Instant app preview on real devices
- **Managed APIs**: Access to sensors without native code

Set Up Your Environment

Set up a local development environment for running your project on iOS and Android

- docs.expo.dev/get-started/set-up-your-environment/
Install
- Node.js (Latest LTS version): nodejs.org/en/download
- Expo Go app: Available on App Store and Google Play
- iOS simulator (**Xcode**)
- Android emulator (**Android Studio**)

Expo Workflow

. Initialize project:

```
npx create-expo-app@latest
```

? Run locally:

```
cd my-app  
npx expo start
```

} Preview:

- Expo Go: Scan QR code on device
- Simulator/Emulator: Press [i](#) (iOS) or [a](#) (Android)

Make First Change

my-app/app/(tabs)/index.tsx

```
<ThemedView style={styles.titleContainer}>
  <ThemedText type="title">Welcome!</ThemedText>
  <HelloWave />
</ThemedView>
```

Hot Reloading: Automatically update app on code change

Expo Go

Enables testing on real devices without complex builds

Build: Process of packaging your app into an **installable file** (`.ipa` for iOS, `.apk` for Android)

- Compile native code (Swift/Obj-C, Java/Kotlin)
- Bundle JavaScript and assets
- Sign the app so it can run on a real device
- Attach a digital signature

Expo Go

Enables testing on real devices without complex builds

Build: Process of packaging your app into an **installable file** (`.ipa` for iOS, `.apk` for Android)

Expo Go: Pre-built container app (“pre-installed testbed”)

- With React Native runtime and common native modules
- Skip the build step for development
- Run your app instantly by scanning a QR code
- No need for Xcode or Android Studio setup

React Native + Expo: Benefits

- **Cross-Platform:** ~90% code reuse between iOS and Android
- **Rapid Development:** Expo's managed workflow speeds up prototyping
- **Sensor Access:** Build creative apps using camera, gyroscope, etc.
- **Ecosystem:** Rich libraries, tutorials, and community support

Your First Expo Project

```
npx create-expo-app@latest
```

Reset to start fresh with a new project

```
npm run reset-project
```

Start Fresh

my-app/app/index.tsx

```
1 import { Text, View } from "react-native";
2
3 export default function Index() {
4   return (
5     <View
6       style={{
7         flex: 1,
8         justifyContent: "center",
9         alignItems: "center",
10      }}
11     >
12       <Text>Edit app/index.tsx to edit this screen.</Text>
13     </View>
14   );
15 }
```

Add Button

my-app/app/index.tsx

```
1 import { Button, Text, View } from "react-native";
2
3 export default function Index() {
4   return (
5     <View
6       style={{
7         flex: 1,
8         justifyContent: "center",
9         alignItems: "center",
10      }}
11     >
12       <Text>Edit app/index.tsx to edit this screen.</Text>
13       <Button title="Click me" onPress={() => alert("Hello!")} />
14     </View>
15   );
16 }
```

Recap: Mobile Dev Setup

React Native: JavaScript-based framework for building native-like apps for iOS and Android

Expo: Simplifies React Native development

- Workflow
- Initialize with `npx create-expo-app@latest`
- Run with `npx expo start`
- Preview with Expo Go, iOS Simulator, Android Emulator
- Benefits: Rapid prototyping, sensor access, strong community

Lecture Summary

React Refresher

- Declarative UI updates for interactive apps
- Components, props, hooks (`useState`, `useEffect`)

React Native: Cross-platform native apps with one JavaScript codebase

Expo: Simplifies setup, testing, and sensor access

- Workflow: Initialize, run, and preview with hot reloading

Speaker notes